

1. 项目概述

本项目基于 Spring Boot 3.x + Redis 实现短信验证码登录功能，使用容联云短信服务发送验证码，Redis 存储验证码并设置过期时间。

2. 功能特性

- 手机号格式校验
- 短信验证码发送（集成容联云短信服务）
- Redis 验证码存储和过期管理
- 验证码登录认证
- Token 生成和返回
- 跨域配置支持
- 完善的异常处理

3. 添加依赖

版本由父项目统一通过 BOM 管理。

```
1  <dependencies>
2      <!-- Spring Boot Starter Data Redis -->
3      <dependency>
4          <groupId>org.springframework.boot</groupId>
5          <artifactId>spring-boot-starter-data-redis</artifactId>
6      </dependency>
7
8      <!-- FastJson2 -->
9      <dependency>
10         <groupId>com.alibaba.fastjson2</groupId>
11         <artifactId>fastjson2</artifactId>
12     </dependency>
13
14     <!-- 容联云短信 SDK -->
15     <dependency>
16         <groupId>com.cloopen</groupId>
17         <artifactId>java-sms-sdk</artifactId>
18     </dependency>
19 </dependencies>
```

4. 配置文件

```

1  server:
2      port: 8888
3
4  spring:
5      # Redis 配置
6      data:
7          redis:
8              # Redis 服务器地址
9              host: localhost
10             # Redis 服务器端口
11             port: 6379
12             # 连接密码（如果设置了密码）
13             password:
14             # 连接超时时间（毫秒）
15             timeout: 5000ms
16             # 数据库索引（0-15，默认为0）
17             database: 0
18             # 连接池配置（使用 Lettuce）
19             lettuce:
20                 pool:
21                     # 连接池最大连接数
22                     max-active: 200
23                     # 连接池最大阻塞等待时间（负数表示没有限制）
24                     max-wait: -1ms
25                     # 连接池中的最大空闲连接
26                     max-idle: 10
27                     # 连接池中的最小空闲连接
28                     min-idle: 0
29                     # 空闲连接检测周期
30                     time-between-eviction-runs: 60s
31
32  mqxu:
33      sms:
34      ccp:
35          serverIp: app.cloopen.com
36          port: 8883
37          accountSid: 你的容联云accountSid
38          accountToken: 你的容联云accountToken
39          appId: 你的容联云appId
40          templateId: 1

```

5. 目录结构

```

1  spring-boot-redis/
2  |—— src/main/java/top/mqxu/boot/redis/
3  |   |—— cache/
4  |   |   |—— RedisCache.java           # Redis缓存工具
5  |   |   |—— RedisKeys.java           # Redis键管理
6  |   |—— config/

```

```

7 | | | CorsConfig.java # 跨域配置
8 | | | RedisConfig.java # Redis配置
9 | | | CloopenConfig.java # 容联云配置
10 | | controller/
11 | | | SmsController.java # 短信发送控制器
12 | | | LoginController.java # 登录控制器
13 | | dto/
14 | | | LoginRequest.java # 登录请求对象
15 | | enums/
16 | | | ErrorCode.java # 错误码枚举
17 | | exception/
18 | | | ServerException.java # 服务异常
19 | | handler/
20 | | | GlobalExceptionHandler.java # 统一异常处理
21 | | result/
22 | | | Result.java # 统一响应结果
23 | | service/
24 | | | impl/
25 | | | | SmsServiceImpl.java # 短信服务实现
26 | | | | LoginServiceImpl.java # 登录服务实现
27 | | | SmsService.java # 短信服务接口
28 | | | LoginService.java # 登录服务接口
29 | | utils/
30 | | | CommonUtils.java # 通用工具类
31 | | vo/
32 | | | LoginResponse.java # 登录响应对象
33 | | RedisApplication.java # 启动主类
34 | | src/test/ # 测试代码

```

6. 核心代码实现

6.1. 配置类

6.1.1. Redis 配置 (RedisConfig.java)

```
1 package top.mqxu.boot.redis.config;
2
3 import com.alibaba.fastjson2.JSON;
4 import com.alibaba.fastjson2.JSONWriter;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.context.annotation.Configuration;
7 import org.springframework.data.redis.connection.RedisConnectionFactory;
8 import org.springframework.data.redis.core.RedisTemplate;
9 import org.springframework.data.redis.serializer.RedisSerializer;
10 import org.springframework.data.redis.serializer.SerializationException;
11
12 import java.nio.charset.Charset;
13 import java.nio.charset.StandardCharsets;
14
```

```

15  /**
16   * Redis配置类,配置RedisTemplate和自定义序列化器
17   */
18  @Configuration
19  public class RedisConfig {
20
21      /**
22       * 配置RedisTemplate
23       * 使用FastJson2作为序列化器, 提高性能并保证序列化的可读性
24       *
25       * @param factory Redis连接工厂
26       * @return 配置好的RedisTemplate
27       */
28      @Bean
29      public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory) {
30          RedisTemplate<String, Object> template = new RedisTemplate<>();
31          // 设置key的序列化器为String序列化器, 这样Redis中的key就是可读的字符串形式
32          template.setKeySerializer(RedisSerializer.string());
33          // 设置Hash类型的key序列化器为String序列化器
34          template.setHashKeySerializer(RedisSerializer.string());
35          // 创建FastJson2序列化器实例
36          FastJsonRedisSerializer<Object> serializer = new FastJsonRedisSerializer<>
37              (Object.class);
38          // 设置value的序列化器为FastJson2序列化器, 支持Java对象与JSON的高效转换
39          template.setValueSerializer(serializer);
40          // 设置Hash类型的value序列化器
41          template.setHashValueSerializer(serializer);
42          // 设置Redis连接工厂
43          template.setConnectionFactory(factory);
44          return template;
45      }
46
47      /**
48       * FastJson2 Redis序列化器
49       * 使用FastJson2进行对象的序列化和反序列化
50       * 相比默认的JDK序列化器, 具有更好的性能和可读性
51       *
52       * @param <T> 序列化的对象类型
53       */
54      record FastJsonRedisSerializer<T>(Class<T> clazz) implements RedisSerializer<T> {
55          /**
56           * 默认字符集: UTF-8
57           */
58          public static final Charset DEFAULT_CHARSET = StandardCharsets.UTF_8;
59
60          /**
61           * 序列化: 将Java对象转换为字节数组
62           *
63           * @param t 要序列化的对象
64           * @return 序列化后的字节数组
65           * @throws SerializationException 序列化异常
66           */

```

```

66         @Override
67         public byte[] serialize(T t) throws SerializationException {
68             if (null == t) {
69                 return new byte[0];
70             }
71             // 使用FastJson2将对象转为JSON字符串
72             // WriteClassName特性会在JSON中包含类名信息, 便于反序列化时确定类型
73             return JSON.toJSONString(t,
JSONWriter.Feature.WriteClassName).getBytes(DEFAULT_CHARSET);
74         }
75
76         /**
77          * 反序列化: 将字节数组转换为Java对象
78          *
79          * @param bytes 要反序列化的字节数组
80          * @return 反序列化后的Java对象
81          * @throws SerializationException 反序列化异常
82          */
83         @Override
84         public T deserialize(byte[] bytes) throws SerializationException {
85             if (null == bytes || bytes.length == 0) {
86                 return null;
87             }
88             // 将字节数组转换为字符串
89             String str = new String(bytes, DEFAULT_CHARSET);
90             // 使用FastJson2将JSON字符串转换为Java对象
91             return JSON.parseObject(str, clazz);
92         }
93     }
94 }

```

6.1.2. 短信配置 (CloopenConfig.java)

```

1  package top.mqxu.boot.redis.config;
2
3  import lombok.Data;
4  import org.springframework.boot.context.properties.ConfigurationProperties;
5  import org.springframework.context.annotation.Configuration;
6
7  /**
8   * 短信配置
9   */
10 @Data
11 @Configuration
12 @ConfigurationProperties(prefix = "mqxu.sms.ccp")
13 public class CloopenConfig {
14     private String serverIp;
15     private String port;
16     private String accountSid;
17     private String accountToken;
18     private String appId;
19     private String templateId;

```

```
20 }
```

6.1.3. 跨域配置 (CorsConfig.java)

```
1  package top.mqxu.boot.redis.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.web.cors.CorsConfiguration;
6  import org.springframework.web.cors.CorsConfigurationSource;
7  import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
8  import org.springframework.web.servlet.config.annotation.CorsRegistry;
9  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
10
11  /**
12   * 跨域配置
13   */
14  @Configuration
15  public class CorsConfig implements WebMvcConfigurer {
16
17      @Override
18      public void addCorsMappings(CorsRegistry registry) {
19          registry.addMapping("/**")
20              .allowedOrigins("*")
21              .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
22              .allowedHeaders("*")
23              .maxAge(3600);
24      }
25
26      @Bean
27      public CorsConfigurationSource corsConfigurationSource() {
28          CorsConfiguration configuration = new CorsConfiguration();
29          configuration.addAllowedOrigin("*");
30          configuration.addAllowedMethod("*");
31          configuration.addAllowedHeader("*");
32          configuration.setMaxAge(3600L);
33
34          UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
35          source.registerCorsConfiguration("/**", configuration);
36          return source;
37      }
38  }
```

说明：

- 允许所有域名跨域访问
- 支持常用 HTTP 方法
- 预检请求缓存 3600 秒

6.2. Redis 缓存工具类

6.2.1. RedisCache

```

1  package top.mqxu.boot.redis.cache;
2
3
4  import jakarta.annotation.Resource;
5  import org.springframework.data.redis.core.HashOperations;
6  import org.springframework.data.redis.core.RedisTemplate;
7  import org.springframework.stereotype.Component;
8
9  import java.util.Collection;
10 import java.util.Map;
11 import java.util.concurrent.TimeUnit;
12
13 /**
14  * Redis缓存工具类
15  */
16 @Component
17 public class RedisCache {
18     /**
19      * 默认过期时长为24小时，单位：秒
20      */
21     public final static long DEFAULT_EXPIRE = 60 * 60 * 24L;
22     /**
23      * 过期时长为1小时，单位：秒
24      */
25     public final static long HOUR_ONE_EXPIRE = (long) 60 * 60;
26     /**
27      * 过期时长为6小时，单位：秒
28      */
29     public final static long HOUR_SIX_EXPIRE = 60 * 60 * 6L;
30     /**
31      * 不设置过期时长
32      */
33     public final static long NOT_EXPIRE = -1L;
34
35     @Resource
36     private RedisTemplate<String, Object> redisTemplate;
37
38     public void set(String key, Object value, long expire) {
39         redisTemplate.opsForValue().set(key, value);
40         if (expire != NOT_EXPIRE) {
41             expire(key, expire);
42         }
43     }
44
45     public void set(String key, Object value) {
46         set(key, value, DEFAULT_EXPIRE);
47     }
48
49     public Object get(String key, long expire) {
50         Object value = redisTemplate.opsForValue().get(key);

```

```

51         if (expire != NOT_EXPIRE) {
52             expire(key, expire);
53         }
54         return value;
55     }
56
57     public Object get(String key) {
58         return get(key, NOT_EXPIRE);
59     }
60
61     public Long increment(String key) {
62         return redisTemplate.opsForValue().increment(key);
63     }
64
65     public Boolean hasKey(String key) {
66         return redisTemplate.hasKey(key);
67     }
68
69     public void delete(String key) {
70         redisTemplate.delete(key);
71     }
72
73     public void delete(Collection<String> keys) {
74         redisTemplate.delete(keys);
75     }
76
77     public Object hGet(String key, String field) {
78         return redisTemplate.opsForHash().get(key, field);
79     }
80
81     public Map<String, Object> hGetAll(String key) {
82         HashOperations<String, String, Object> hashOperations =
redisTemplate.opsForHash();
83         return hashOperations.entries(key);
84     }
85
86     public void hMSet(String key, Map<String, Object> map) {
87         hMSet(key, map, DEFAULT_EXPIRE);
88     }
89
90     public void hMSet(String key, Map<String, Object> map, long expire) {
91         redisTemplate.opsForHash().putAll(key, map);
92
93         if (expire != NOT_EXPIRE) {
94             expire(key, expire);
95         }
96     }
97
98     public void hSet(String key, String field, Object value) {
99         hSet(key, field, value, DEFAULT_EXPIRE);
100     }
101

```



```

102     public void hSet(String key, String field, Object value, long expire) {
103         redisTemplate.opsForHash().put(key, field, value);
104
105         if (expire != NOT_EXPIRE) {
106             expire(key, expire);
107         }
108     }
109
110     public void expire(String key, long expire) {
111         redisTemplate.expire(key, expire, TimeUnit.SECONDS);
112     }
113
114     public void hDel(String key, Object... fields) {
115         redisTemplate.opsForHash().delete(key, fields);
116     }
117
118     public void leftPush(String key, Object value) {
119         leftPush(key, value, DEFAULT_EXPIRE);
120     }
121
122     public void leftPush(String key, Object value, long expire) {
123         redisTemplate.opsForList().leftPush(key, value);
124
125         if (expire != NOT_EXPIRE) {
126             expire(key, expire);
127         }
128     }
129
130     public Object rightPop(String key) {
131         return redisTemplate.opsForList().rightPop(key);
132     }
133 }

```

6.2.2. RedisKeys

```

1  package top.mqxu.boot.redis.cache;
2
3  /**
4   * @Author: mxqu
5   * @Date: 2025/9/24
6   */
7  public class RedisKeys {
8
9      /**
10       * 验证码 Key
11       */
12     public static String getSmsKey(String phone) {
13         return "sms:captcha:" + phone;
14     }
15 }

```

6.3. 扩充错误枚举

可以适当扩充错误枚举类，加上我们自己的业务异常，如：

```

1  @AllArgsConstructor
2  @Getter
3  public enum ErrorCode {
4      BAD_REQUEST(400, "请求参数不合法"),
5      UNAUTHORIZED(401, "登录失效, 请重新登录"),
6      FORBIDDEN(403, "无权限访问"),
7      NOT_FOUND(404, "资源不存在"),
8      INTERNAL_SERVER_ERROR(500, "服务器内部错误, 请稍后再试"),
9
10     PHONE_ERROR(40001, "手机号格式错误"),
11     PHONE_NOT_EXIST(40002, "手机号不存在"),
12     PHONE_CODE_EMPTY(40003, "验证码不能为空"),
13     PHONE_CODE_ERROR(40004, "验证码错误"),
14     PHONE_CODE_EXPIRED(40005, "验证码不存在或已过期"),
15     PHONE_CODE_USED(40006, "验证码已使用"),
16     PHONE_CODE_SEND_TOO_MANY(40007, "验证码发送过于频繁"),
17     PHONE_CODE_SEND_FAILED(40008, "验证码发送失败"),
18     PHONE_CODE_SEND_LIMIT(40009, "验证码发送次数已达上限");
19
20
21     private final int code;
22     private final String msg;
23 }

```

6.4. 省略

省略自定义异常类 `ServerException`、统一异常处理 `GlobalExceptionHandler`、统一响应返回 `Result`。

6.5. 工具类

utils 子包, `CommonUtils.java`

```

1  package top.mqxu.boot.redis.utils;
2
3  import java.util.concurrent.ThreadLocalRandom;
4  import java.util.regex.Matcher;
5  import java.util.regex.Pattern;
6
7  public class CommonUtils {
8
9      /**
10       * 校验手机号码
11       *
12       * @param phone 电话
13       * @return boolean
14       */
15     public static boolean checkPhone(String phone) {

```

```

16         if (phone == null || phone.length() != 11) {
17             return false;
18         }
19         // 中国大陆手机号码的正则表达式，可能需要根据实际情况进行调整
20         String regex = "^1[3-9]\\d{9}$";
21         Pattern pattern = Pattern.compile(regex);
22         Matcher matcher = pattern.matcher(phone);
23         return matcher.matches();
24     }
25
26     /**
27      * 生成4位随机验证码
28      *
29      * @return int
30      */
31     public static int generateCode() {
32         return ThreadLocalRandom.current().nextInt(1000, 9999);
33     }
34
35 }

```

6.6. DTO/VO

6.6.1. 登录请求对象 (LoginRequest.java)

```

1  @Data
2  public class LoginRequest {
3      private String phone;
4      private String code;
5  }

```

6.6.2. 登录响应对象 (LoginResponse.java)

```

1  @Data
2  @NoArgsConstructor
3  @AllArgsConstructor
4  public class LoginResponse {
5      private String token;
6      private String phone;
7      private String message;
8
9      public LoginResponse(String token, String phone) {
10         this.token = token;
11         this.phone = phone;
12         this.message = "登录成功";
13     }
14 }

```

6.7. 服务层

6.7.1. 短信服务接口 (SmsService.java)

```

1  public interface SmsService {
2
3      /**
4       * 发送短信
5       *
6       * @param phone 手机号
7       */
8      boolean sendSms(String phone);
9  }

```

6.7.2. 短信服务实现 (SmsServiceImpl.java)

```

1  @Slf4j
2  @Service
3  @AllArgsConstructor
4  public class SmsServiceImpl implements SmsService {
5      private final CloopenConfig cloopenConfig;
6      private final RedisCache redisCache;
7
8      /**
9       * 发送短信验证码
10      *
11      * @param phone 手机号
12      * @return 发送成功返回true, 失败返回false
13      */
14      @Override
15      public boolean sendSms(String phone) {
16          // 1.校验手机号
17          if (!CommonUtils.checkPhone(phone)) {
18              throw new ServerException(ErrorCode.PHONE_ERROR);
19          }
20          // 2.生成验证码, 并存入 Redis, 60秒有效
21          int code = CommonUtils.generateCode();
22          redisCache.set(RedisKeys.getSmsKey(phone), code, 60);
23          log.info("发送短信验证码: {}", code);
24          boolean flag = true;
25          flag = send(phone, code);
26          return flag;
27      }
28
29      /**
30       * 发送短信验证码
31       *
32       * @param phone 手机号
33       * @param code 验证码
34       * @return 发送成功返回true, 失败返回false
35       */
36      private boolean send(String phone, int code) {
37          // 2.获取到配置信息

```

```

38     String serverIp = cloopenConfig.getServerIp();
39     String serverPort = cloopenConfig.getPort();
40     String accountSid = cloopenConfig.getAccountSid();
41     String accountToken = cloopenConfig.getAccountToken();
42     String appId = cloopenConfig.getAppId();
43     String templateId = cloopenConfig.getTemplateId();
44
45     // 3.创建sdk对象
46     CCPRestSmsSDK sdk = new CCPRestSmsSDK();
47     sdk.init(serverIp, serverPort);
48     sdk.setAccount(accountSid, accountToken);
49     sdk.setAppId(appId);
50     sdk.setBodyType(BodyType.Type_JSON);
51     String[] datas = {String.valueOf(code), "1"};
52     // 发送短信
53     HashMap<String, Object> result = sdk.sendTemplateSMS(phone, templateId, datas,
"1234", UUID.randomUUID().toString());
54     if ("000000".equals(result.get("statusCode"))) {
55         // 正常返回输出data包体信息 (map)
56         HashMap<String, Object> data = (HashMap<String, Object>) result.get("data");
57         Set<String> keySet = data.keySet();
58         for (String key : keySet) {
59             Object object = data.get(key);
60             log.info("{} = {}", key, object);
61         }
62     } else {
63         // 异常返回输出错误码和错误信息
64         log.error("错误码={} 错误信息= {}", result.get("statusCode"),
result.get("statusMsg"));
65         return false;
66     }
67     return true;
68 }
69 }
70

```

6.7.3. 登录服务接口 (LoginService.java)

```

1  public interface LoginService {
2
3      /**
4       * 验证码登录
5       *
6       * @param loginRequest 登录请求参数
7       * @return 登录响应
8       */
9      LoginResponse login(LoginRequest loginRequest);
10 }

```

6.7.4. 登录服务实现 (LoginServiceImpl.java)

```

1  @Slf4j
2  @Service
3  @AllArgsConstructor
4  public class LoginServiceImpl implements LoginService {
5      private final RedisCache redisCache;
6
7      @Override
8      public LoginResponse login(LoginRequest loginRequest) {
9          String phone = loginRequest.getPhone();
10         String inputCode = loginRequest.getCode();
11
12         // 1.校验手机号格式
13         if (!CommonUtils.checkPhone(phone)) {
14             throw new ServerException(ErrorCode.PHONE_ERROR);
15         }
16
17         // 2.校验验证码是否为空
18         if (inputCode == null || inputCode.trim().isEmpty()) {
19             throw new ServerException(ErrorCode.PHONE_CODE_EMPTY);
20         }
21
22         // 3.从Redis中获取验证码
23         String redisKey = RedisKeys.getSmsKey(phone);
24         String redisCode = redisCache.get(redisKey).toString();
25
26         // 4.验证码不存在或已过期
27         if (redisCode == null) {
28             throw new ServerException(ErrorCode.PHONE_CODE_EXPIRED);
29         }
30
31         // 5.验证码不匹配
32         if (!inputCode.equals(redisCode)) {
33             throw new ServerException(ErrorCode.PHONE_CODE_ERROR);
34         }
35
36         // 6.验证成功，删除Redis中的验证码
37         redisCache.delete(redisKey);
38
39         // 7.生成token并返回登录信息
40         String token = generateToken(phone);
41
42         log.info("用户 {} 登录成功", phone);
43         return new LoginResponse(token, phone);
44     }
45
46     private String generateToken(String phone) {
47         return UUID.randomUUID().toString().replace("-", "") + phone.hashCode();
48     }
49 }

```

6.8. 控制器层

6.8.1. 短信控制器 (SmsController.java)

```
1  @RestController
2  @RequestMapping("/api")
3  public class SmsController {
4      @Resource
5      private SmsService smsService;
6
7      @GetMapping("/sms/send")
8      public Result<Boolean> sendSms(@RequestParam String phone) {
9          return Result.ok(smsService.sendSms(phone));
10     }
11 }
```

6.8.2. 登录控制器 (LoginController.java)

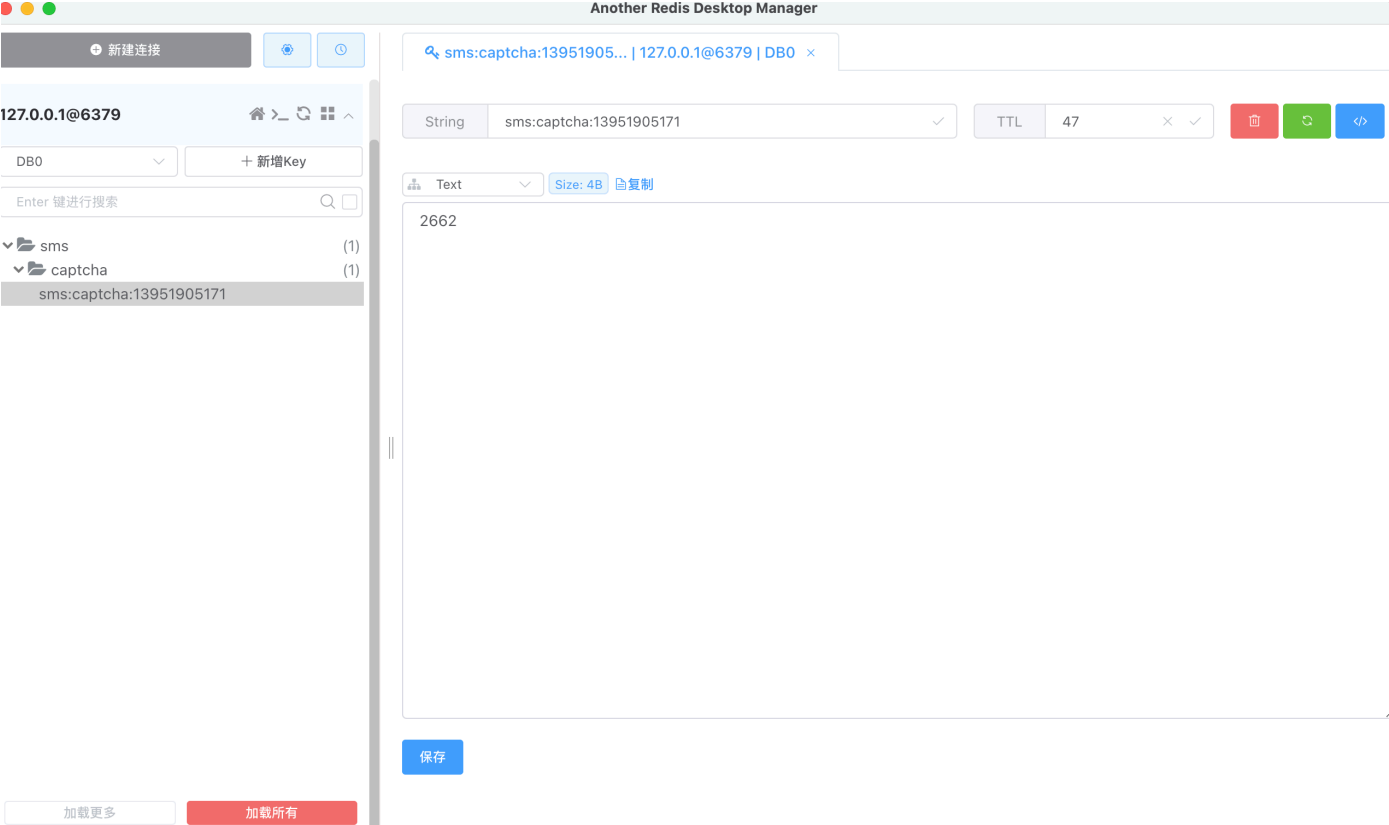
```
1  @RestController
2  @RequestMapping("/api")
3  @AllArgsConstructor
4  public class LoginController {
5      private final LoginService loginService;
6
7      @PostMapping("/login")
8      public Result<LoginResponse> login(@RequestBody LoginRequest loginRequest) {
9          LoginResponse loginResponse = loginService.login(loginRequest);
10         return Result.ok(loginResponse);
11     }
12 }
```

6.9. 接口测试

验证码发送成功



Redis 中存入了验证码



手机号不符合要求，抛出自定义异常

spring-boot-redis (1)

- top.mqxu.boot.redis.contr
 - Sms (1)
 - sendSms** /sms

调试

GET 发送

Query Body Path Headers Cookies

☐	参数名	参数值	
☒	phone	139519051	🗑
☒			🗑

**手机号没有通过校验
抛出自定义异常**

Body Cookies Headers

🛒 ↺ ⬇ JSON 200 8 ms 49 B

```
1 {
2   "code": 3003,
3   "msg": "手机号错误",
4   "data": null
5 }
```

7. 安全考虑

7.1. 验证码安全

- 验证码 60 秒过期
- 验证成功后立即删除
- 手机号格式严格校验

7.2. 接口安全

- 统一异常处理
- 参数校验
- 跨域配置

7.3. 生产环境建议

- 添加验证码发送频率限制
- 添加 IP 限流
- 使用更安全的 Token 生成算法
- 配置 HTTPS